

Written_Homework3

Jeffrey Ochavillo

2023-05-10

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.2      ✓ readr      2.1.4
## ✓ forcats   1.0.0      ✓ stringr   1.5.0
## ✓ ggplot2   3.4.2      ✓ tibble    3.2.1
## ✓ lubridate 1.9.2      ✓ tidyr     1.3.0
## ✓ purrr     1.0.1
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

```
library(keras)
library(e1071)
library(dplyr)
library(abind)
```

Load the data

```

amecro <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/amecro.dat")
amecro=species
barswa <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/barswa.dat")
barswa=species
bkcchi <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/bkcchi.dat")
bkcchi=species
blujay <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/blujay.dat")
blujay=species
daejun <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/daejun.dat")
daejun=species
houfin <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/houfin.dat")
houfin=species
mallar3 <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/mallar3.dat")
mallar3=species
norfli <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/norfli.dat")
norfli=species
rewbla <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/rewbla.dat")
rewbla=species
stejay <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/stejay.dat")
stejay = species
wesmea <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/wesmea.dat")
wesmea=species
whcspa <- load("/Users/jeffreyochavillo/DATA 5322/spectrograms/whcspa.dat")
whcspa=species

```

```
library(dplyr)
```

```

w3_data <- abind(amecro, barswa, bkcchi, blujay, daejun, houfin, mallar3, norfli, rewbl
a, stejay, wesmea, whcspa, along =1)
dim(w3_data)

```

```
## [1] 580 343 256
```

```

x <- abind(blujay, stejay, along = 1)
#x <- abind(x, along = 4)
x <- array_reshape(x, c(nrow(x),87808))
y <- c(rep('blujay',dim(blujay)[1]), rep('stejay',dim(stejay)[1]))
y_factor <- factor(y,level = c('blujay','stejay'))
y_encode <- to_categorical(as.integer(y_factor)-1, num_classes = 2)
# y_encode
# glimpse(y)
# dim(w3_data)
# glimpse(w3_data)
# dim(x)

```

```
set.seed(42)
library(keras)
library(abind)
w3_data <- abind(amecro, barswa, bkcchi, blujay, daejun, houfin, mallar3, norfli, rewbl
a, stejay, wesmea, whcspa, along =1)
dim(w3_data)
```

```
## [1] 580 343 256
```

```

# Combine the spectrograms into one matrix
x <- abind(blujay, stejay, along = 1)
y <- c(rep('blujay', dim(blujay)[1]), rep('stejay', dim(stejay)[1]))

# length(y)
# Create the target variable (labels)

x <- array_reshape(x, c(nrow(x), 87808))
y_factor <- factor(y, levels = c('blujay', 'stejay'))
#length(y_factor)
y_encode <- to_categorical(as.integer(y_factor) - 1, num_classes = 2)
#length(y_encode)

# Split the data into training and testing sets
n <- nrow(x)
train_measure <- sample(n, floor(0.8*n))

x_train <- x[train_measure,]
y_train <- y_encode[train_measure, ]
x_test <- x[-train_measure,]
y_test <- y_encode[-train_measure,]

# Define the model
# model <- keras_model_sequential()
# model %>%
#   layer_dense(units = 64, activation = "relu", input_shape = c(87808)) %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 32, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 2, activation = "softmax") # Output layer with softmax activation for binary classification #this model took less than 30 seconds to run

# > cat("Test loss:", metrics[[1]], "\n")
# Test loss: 0.2810534
# > cat("Test accuracy:", metrics[[2]], "\n")
# Test accuracy: 0.8888889

# > cat("Test loss:", metrics[[1]], "\n")
# Test loss: 0.2052964
# > cat("Test accuracy:", metrics[[2]], "\n")
# Test accuracy: 0.9444444

model <- keras_model_sequential()
model %>%
  layer_dense(units = 128, activation = "relu", input_shape = c(87808)) %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 64, activation = "relu") %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dropout(rate = 0.5) %>%

```

```

layer_dense(units = 2, activation = "softmax") # Output layer with softmax activation
for binary classification

# > cat("Test loss:", metrics[[1]], "\n")
# Test loss: 0.3542859
# > cat("Test accuracy:", metrics[[2]], "\n")
# Test accuracy: 0.7222222 also took less than 30 seconds to run

# > cat("Test loss:", metrics[[1]], "\n")
# Test loss: 0.5489867
# > cat("Test accuracy:", metrics[[2]], "\n")
# Test accuracy: 0.8888889

# Compile the model
model %>% compile(
  loss = "categorical_crossentropy",
  optimizer = "adam",
  metrics = c("accuracy")
)

# Train the model
model %>% fit(
  x = x_train,
  y = y_train,
  epochs = 10,
  batch_size = 32,
  validation_data = list(x_test, y_test)
)

# Evaluate the model
metrics <- model %>% evaluate(x_test, y_test)
cat("Test loss:", metrics[[1]], "\n")

```

```
## Test loss: 0.3532373
```

```
cat("Test accuracy:", metrics[[2]], "\n")
```

```
## Test accuracy: 0.8333333
```

classification for all 12 species

```

# Example variables to encode
variables <- c("var1", "var2", "var3", "var4", "var5", "var6", "var7", "var8", "var9",
"var10", "var11", "var12")

# Example labels (numeric)
labels <- c("amecro", "barswa", "bkcchi", "blujay", "daejun", "houfin", "mallar3", "norf
li", "rewbla", "stejay", "wesmea", "whcspa")

# Create a data frame with the variables
data <- data.frame(var1 = c(1, 2, 3),
                    var2 = c(1, 3, 2),
                    var3 = c(2, 1, 3),
                    var4 = c(3, 2, 1),
                    var5 = c(2, 3, 1),
                    var6 = c(3, 1, 2),
                    var7 = c(1, 2, 3),
                    var8 = c(2, 3, 1),
                    var9 = c(3, 1, 2),
                    var10 = c(1, 2, 3),
                    var11 = c(2, 3, 1),
                    var12 = c(3, 1, 2))

# One-hot encode the variables
encoded_data <- model.matrix(~ . - 1, data = data)

# Rename the encoded columns with labels
colnames(encoded_data) <- labels

# View the encoded data
print(encoded_data)

```

```

##   amecro barswa bkcchi blujay daejun houfin mallar3 norfli rewbla stejay wesmea
## 1      1      1      2      3      2      3      1      2      3      1      2
## 2      2      3      1      2      3      1      2      3      1      2      3
## 3      3      2      3      1      1      2      3      1      2      3      1
##   whcspa
## 1      3
## 2      1
## 3      2
## attr(,"assign")
## [1] 1 2 3 4 5 6 7 8 9 10 11 12

```

```
# Combine the spectrograms into a single array
spectrograms <- array(0, dim = c(594, 343, 256))
spectrograms[1:52,,] <- amecro
spectrograms[53:107,,] <- barswa
spectrograms[108:164,,] <- bkcchi
spectrograms[165:214,,] <- blujay
spectrograms[215:272,,] <- daejun
spectrograms[273:317,,] <- houfin
spectrograms[318:353,,] <- mallar3
spectrograms[354:412,,] <- norfli
spectrograms[413:453,,] <- rewbla
spectrograms[454:493,,] <- stejay
spectrograms[494:529,,] <- wesmea
spectrograms[530:580,,] <- whcspa

# Label the data
labels <- c(rep("amecro", 52), rep("barswa", 55), rep("bkcchi", 57), rep("blujay", 50),
            rep("daejun", 58), rep("houfin", 45), rep("mallar3", 36), rep("norfli", 59),
            rep("rewbla", 41), rep("stejay", 40), rep("wesmea", 36), rep("whcspa", 51))

# Shuffle the data and labels
index <- sample(1:length(labels))
spectrograms <- spectrograms[index,,]
labels <- labels[index]

# Create a training and testing set
train_size <- round(length(labels) * 0.7)
train_data <- spectrograms[1:train_size,,]
train_labels <- labels[1:train_size]
test_data <- spectrograms[(train_size+1):length(labels),,]
test_labels <- labels[(train_size+1):length(labels)]

dim(train_data)
```

```
## [1] 406 343 256
```

```
dim(test_data)
```

```
## [1] 174 343 256
```

```
# reshape the data
train_data <- array_reshape(train_data, c(train_size, 343, 256, 1))
test_data <- array_reshape(test_data, c(length(labels) - train_size, 343, 256, 1))

# one hot encode the labels
# train_labels <- one_hot(train_labels, 12)
# test_labels <- one_hot(test_labels, 12)

# Convert labels to numerical categories
label_categories <- factor(labels)
label_categories <- as.integer(label_categories) - 1

# One-hot encode the labels using to_categorical
train_labels <- to_categorical(label_categories[1:train_size], num_classes = 12)
test_labels <- to_categorical(label_categories[(train_size+1):length(labels)], num_classes = 12)

dim(train_data)
```

```
## [1] 406 343 256 1
```

```
dim(test_data)
```

```
## [1] 174 343 256 1
```



```

library(keras)

# Define the model architecture

model <- keras_model_sequential() %>%
  layer_conv_2d(filters = 32, kernel_size = c(3, 3), activation = "relu", input_shape =
c(343, 256, 1)) %>%
  layer_max_pooling_2d(pool_size = c(2, 2)) %>%
  layer_flatten() %>%
  layer_dense(units = 64, activation = "relu") %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 12, activation = "softmax") #7 min run and Test accuracy: 0.643678
2

# model <- keras_model_sequential() %>%
#   layer_conv_2d(filters = 32, kernel_size = c(3, 3), activation = "relu", input_shape
= c(343, 256, 1)) %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_conv_2d(filters = 64, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_conv_2d(filters = 128, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_flatten() %>%
#   layer_dense(units = 128, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 12, activation = "softmax") Test accuracy: 0.7241379

# model <- keras_model_sequential() %>%
#   layer_conv_2d(filters = 64, kernel_size = c(3, 3), activation = "relu", input_shape
= c(343, 256, 1)) %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_conv_2d(filters = 128, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_conv_2d(filters = 256, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_flatten() %>%
#   layer_dense(units = 256, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 128, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 12, activation = "softmax") #Test accuracy: 0.1091954 took 34 mi
nutes to run

#model <- keras_model_sequential() %>%
#   layer_conv_2d(filters = 32, kernel_size = c(3, 3), activation = "relu", input_shape =
c(343, 256, 1)) %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%

```

```
# layer_flatten() %>%
# layer_dense(units = 64, activation = "relu") %>%
# layer_dropout(rate = 0.5) %>%
# layer_dense(units = 12, activation = "softmax") #7 min run and Test accuracy: 0.64367
82

# model <- keras_model_sequential() %>%
#   layer_conv_2d(filters = 64, kernel_size = c(3, 3), activation = "relu", input_shape
# = c(343, 256, 1)) %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_conv_2d(filters = 128, kernel_size = c(3, 3), activation = "relu") %>%
#   layer_max_pooling_2d(pool_size = c(2, 2)) %>%
#   layer_batch_normalization() %>%
#   layer_flatten() %>%
#   layer_dense(units = 128, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 64, activation = "relu") %>%
#   layer_dropout(rate = 0.5) %>%
#   layer_dense(units = 12, activation = "softmax") #25 minute run with 0.05 accuracy

# Compile the model
model %>% compile(
  loss = "categorical_crossentropy",
  optimizer = optimizer_rmsprop(lr = 0.001),
  metrics = "accuracy"
)

# Train the model
history <- model %>% fit(
  train_data, train_labels,
  validation_split = 0.2,
  epochs = 20,
  batch_size = 32
)

# # Evaluate the model on test data
scores <- model %>% evaluate(test_data, test_labels)
cat("Test accuracy:", scores[[2]], "\n")
```

```
## Test accuracy: 0.6091954
```